

Aula: Autenticação e Perfil de Usuário em JSP

Objetivo da aula

- Cadastrar usuários
 - Realizar login (autenticação)
 - Trabalhar com sessão
 - Controlar acesso por perfil (autorização)
-

Conceitos importantes

Autenticação vs Autorização

- **Autenticação** → quem é o usuário (login/senha)
- **Autorização** → o que ele pode fazer (perfil)

Exemplo:

- Admin → pode excluir
 - Usuário comum → só visualizar
-

1. Estrutura da tabela

```
CREATE TABLE usuarios (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nome VARCHAR(100),  
    email VARCHAR(100),  
    senha VARCHAR(100),  
    perfil VARCHAR(20)  
);
```

Perfis possíveis:

- ADMIN
 - USER
-

2. Classe Usuario (Model)

```
public class Usuario {  
  
    private int id;  
  
    private String nome;  
  
    private String email;  
  
    private String senha;  
  
    private String perfil;  
  
  
    // getters e setters  
  
}
```

3. DAO – Cadastro e Login

```
public class UsuarioDAO {  
  
    public void cadastrar(Usuario u) {  
  
        String sql = "INSERT INTO usuarios (nome, email, senha, perfil) VALUES (?, ?, ?,  
?)" ;  
  
  
        try (Connection con = ConectaDB.getConexao();  
            PreparedStatement ps = con.prepareStatement(sql)) {  
  
            ps.setString(1, u.getNome());  
            ps.setString(2, u.getEmail());  
            ps.setString(3, u.getSenha());  
            ps.setString(4, u.getPerfil());  
  
  
            ps.execute();  
  
  
        } catch (Exception e) {
```

```

        e.printStackTrace();
    }
}

public Usuario login(String email, String senha) {
    String sql = "SELECT * FROM usuarios WHERE email=? AND senha=?";

    try (Connection con = ConectaDB.getConexao();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, email);
        ps.setString(2, senha);

        ResultSet rs = ps.executeQuery();

        if (rs.next()) {
            Usuario u = new Usuario();
            u.setId(rs.getInt("id"));
            u.setNome(rs.getString("nome"));
            u.setEmail(rs.getString("email"));
            u.setPerfil(rs.getString("perfil"));
            return u;
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

```
        return null;
    }
}
```



4. Tela de Cadastro (JSP)

```
<form action="/CadastrarUsuarioServlet" method="post">
```

```
    Nome: <input type="text" name="nome"><br>
```

```
    Email: <input type="email" name="email"><br>
```

```
    Senha: <input type="password" name="senha"><br>
```

```
    Perfil:
```

```
    <select name="perfil">
```

```
        <option value="USER">Usuário</option>
```

```
        <option value="ADMIN">Administrador</option>
```

```
    </select>
```

```
    <button type="submit">Cadastrar</button>
```

```
</form>
```



5. Servlet de Cadastro

```
@WebServlet("/CadastrarUsuarioServlet")
```

```
public class CadastrarUsuarioServlet extends HttpServlet {
```

```
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) {
```

```
        Usuario u = new Usuario();
```

```
        u.setNome(req.getParameter("nome"));
```

```
        u.setEmail(req.getParameter("email"));
```

```
u.setSenha(req.getParameter("senha"));

u.setPerfil(req.getParameter("perfil"));

UsuarioDAO dao = new UsuarioDAO();
dao.cadastrar(u);

resp.sendRedirect("login.jsp");
}
}
```

6. Tela de Login

```
<form action="LoginServlet" method="post">

    Email: <input type="email" name="email"><br>

    Senha: <input type="password" name="senha"><br>

    <button type="submit">Entrar</button>

</form>
```

7. Servlet de Login

```
@WebServlet("/LoginServlet")

public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req, HttpServletResponse resp) {

        String email = req.getParameter("email");

        String senha = req.getParameter("senha");

        UsuarioDAO dao = new UsuarioDAO();
```

```
Usuario usuario = dao.login(email, senha);

if (usuario != null) {

    HttpSession session = req.getSession();
    session.setAttribute("usuarioLogado", usuario);

    resp.sendRedirect("home.jsp");

} else {
    resp.sendRedirect("login.jsp?erro=1");
}
}
```

8. Usando Sessão (JSP)

```
<%
Usuario u = (Usuario) session.getAttribute("usuarioLogado");

if (u == null) {
    response.sendRedirect("login.jsp");
}
%>
```

Bem-vindo, <%= u.getNome() %>

9. Controle por Perfil

```
<%
```

```
if (u.getPerfil().equals("ADMIN")) {  
    %>  
    <a href="excluir.jsp">Excluir usuários</a>  
    <%  
}  
%>
```

👉 Aqui está o ponto-chave da aula:

- O sistema decide o que mostrar com base no perfil

🔍 O que é um Filter?

Um **Filter** é um componente que intercepta requisições antes de chegarem ao Servlet (ou JSP) — e também pode agir na resposta.

👉 Pense assim:

Requisição → Filter → Servlet → Response

🌱 Para que serve na prática?

Você usa filtros para:

- 🗝️ Autenticação (verificar se usuário está logado)
- 🔒 Autorização (verificar perfil: ADMIN/USER)
- 📊 Log de requisições
- 🛠️ Tratamento de encoding (UTF-8)
- ⌚ Medir tempo de resposta

📌 Estrutura da anotação

```
@WebFilter("/admin/*")
```

Significado:

- O filtro será aplicado a todas as URLs que começam com /admin/
-

🚀 Exemplo completo

```
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebFilter;
import jakarta.servlet.http.*;
import java.io.IOException;

@WebFilter("/admin/*")

public class AuthFilter implements Filter {

    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest request = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) res;

        HttpSession session = request.getSession(false);

        if (session != null && session.getAttribute("usuarioLogado") != null) {
            chain.doFilter(req, res); // continua execução
        } else {
            response.sendRedirect("login.jsp"); // bloqueia acesso
        }
    }
}
```

⚙️ Explicando linha por linha

💠 @WebFilter("/admin/*")

Define onde o filtro atua.

◆ implements Filter

Indica que a classe é um filtro.

◆ doFilter(...)

É o método principal (executado a cada requisição).

◆ request.getSession(false)

- false → não cria sessão nova
 - Apenas verifica se já existe
-

◆ chain.doFilter(req, res)

👉 Continua o fluxo da requisição

Se você **não chamar isso**, a requisição para ali.

◆ response.sendRedirect("login.jsp")

👉 Bloqueia acesso e redireciona

🔒 Exemplo com perfil (mais completo)

```
if (session != null) {  
    Usuario u = (Usuario) session.getAttribute("usuarioLogado");  
  
    if (u != null && u.getPerfil().equals("ADMIN")) {  
        chain.doFilter(req, res);  
        return;  
    }  
}
```

```
response.sendRedirect("login.jsp");
```

👉 Agora não basta estar logado — precisa ser ADMIN.

Mapeamentos possíveis

```
@WebFilter("/*")    // todas requisições
```

```
@WebFilter("/admin/*") // apenas área admin
```

```
@WebFilter("*.jsp")  // apenas arquivos JSP
```

```
@WebFilter("/login") // URL específica
```

Cuidados importantes

Loop infinito

Se proteger login.jsp, você pode travar o acesso.

✓ Solução:

- Ignorar rotas públicas no filtro